Overflow-Detecting and Double-Wide Arithmetic Operations

ISO/IEC JTC1 SC22 WG21 P0103R0 - 2015-09-27

Lawrence Crowl, Lawrence@Crowl.org

Introduction

Some integer arithmetic operations can overflow their representation. Machine architectures generally provide mechanisms to either detect or to construct larger representations in software.

- Machines commonly provide an overflow status, which can be tested in a branch instruction.
- Machines commonly provide a carry status, which can be used with an "add with carry" instruction. The "add with carry" instruction can help implement multi-word addition.
- Machines may provide a double-wide multiply instruction, which provides the upper and lower halves of a multiplication result in separate words. The double-wide multiply instruction can help implement multi-word multiplication.
- Machines may provide a double-wide divide instruction, which provides a dividend twice as wide as the divisor. These can vary in the length of the quotient and whether they return a remainder. The double-wide division instruction can help implement multi-word division.
- Machines may provide a double-wide shift instructions, which provides a result twice as wide as the shifted value. The double-wide shift instructions can help implement scaling for fixed-point arithmetic.

Problem

C and C++ programmers have no standard mechanism to access the machine mechanisms. So, programmers coding within the standards resort to argument pre-checking or subword-emulation. Such code is difficult to get right and inefficient.

More perversely, because C, C++, and Fortran do not provide the mechanisms, machine architects are sometimes no longer including them, which makes some problems difficult so solve efficiently.

Getting compiler support for generating the appropriate instruction sequences is generally easier when the mechanisms span more than one language. This observation means that both C and C++ should support the same mechanism.

Solution

We propose a set of functions that provide the facilities. We do *not* intend that these functions be end-user functions. Our intent is that these functions be tools for library authors to implement types more appropriate to end users.

With use limited to library authors, the syntax can be somewhat more demanding, which in turn means that a syntax suitable for both C and C++ is acceptable. We rely on type-generic macros in C and overloading in C++. Note however, that the operations are applicable beyond built-in types in C++. For larger types, const reference parameters may be more appropriate.

Notation

We use the following operation codes as components of function names.

code	meaning
`cvt`	The value converted to the result type.
`neg`	The negative of the value.

`add`	The sum of the augend and addend.
`add2`	The sum of the augend and two addends. This operation is useful in multiword addition.
`sub`	The difference of minuend and subtrahend.
`sub2`	The difference of the minuend and two subtrahends. This operation is useful in multiword subtraction.
`lsh`	The multiplicand shifted left by the count, i.e. the product of the multiplicand and 2^{count}. The behavior is undefined if the count is less than 0 or if the count greater than or equal to the number of bits in the value type.
`lshadd`	The sum of the multiplicand shifted left by the count and the addend. i.e. the sum of (the product of the multiplicand and 2^{count}) and the addend. The first value shifted left by the count and then sumed with the second value. The behavior is undefined if the count is less than 0 or if the count greater than or equal to the number of bits in the value type. This operation is useful with multiword scaled addition.
`mul`	The product of the multiplier and multiplicand.
`muladd`	The sum of (the product of the multiplicand and the multiplier) and the addend. The behavior is undefined if both the first two values are two's complement minimums. This operation is useful with multiword multiplication.
`muladd2`	The sum of (the product of the multiplicand and the multiplier) and the two addends. The behavior is undefined if either the first two values are two's complement minimums. This operation is useful with multiword multiplication.
`mulsub`	The difference of (the product of the multiplicand and the multiplier) and the subtrahend. The behavior is undefined if both the first two values are two's complement minimums.
`mulsub2`	The difference of (the product of the multiplicand and the multiplier) and the two subtrahend. The behavior is undefined if either the first two values are two's complement minimums.
`divn`	The narrow quotient of the dividend and divisor. The behavior is undefined if the divisor is zero.
`divw`	The wide quotient of the dividend and divisor, The behavior is undefined if the divisor is zero.
`divnrem`	As with `divn` except also computing a remainder with the sign of the divisor.
`divwrem`	As with `divw` except also computing a remainder with the sign of the divisor.

Overflow-Detecting Arithmetic

The overflow-detecting functions return a boolean true when the operation overflows, and a boolean false when the operation does not overflow. Compilers may assume that a true result is rare. When the return is false, the function writes the operation result through the given pointer. When the return is true, the pointer is not used and no write occurs.

The following functions are available. Within these prototypes `T` and `C` are any integer type. However, `C` is useful only when it does not have values that `T` has.

```
bool overflow_cvt( C* result, T value );
bool overflow_neg( T* result, T value );
bool overflow_lsh( T* product, T multiplicand, int count );
bool overflow_add( T* summand, T augend, T addend );
bool overflow_sub( T* difference, T minuend, T subtrahend );
bool overflow_mul( T* product, T multiplicand, T multiplier );
```

Double-Wide Arithmetic

There are two classes of functions, those that provide a result in a single double-wide type and those that provide a result split into two single-wide types.

We expect programmers to use type names from `<cstdin>` or P0102R0 C++ Parametric Number Type Aliases. Hence, we do not need to provide a means to infer one type size from the other. Within this section, we name these types as follows.

`S`	is a signed integer type.
`U`	is an unsigned integer type.
`DS`	is a signed integer type that is double the width of the `S` type.
`DU`	is an unsigned integer type that is double the width of the `U` type.

We need a mechanism to specify the largest supported type for various combinations of function category and operation category. To that end, we propose macros as follows.

macro name	**result category**	**operation category**
`LARGEST_DOUBLE_WIDE_ADD`	double-wide	`add`, `add2`, `sub`, `sub2`

LARGEST_DOUBLE_WIDE_LSH	double-wide	lsh, lshadd
LARGEST_DOUBLE_WIDE_MUL	double-wide	mul, muladd, muladd2, mulsub, mulsub2
LARGEST_DOUBLE_WIDE_DIV	double-wide	divn, divw, divnrem, divwrem
LARGEST_DOUBLE_WIDE_ALL	double-wide	the minimum size of the four macros above
LARGEST_SINGLE_WIDE_ADD	single-wide	add, add2, sub, sub2
LARGEST_SINGLE_WIDE_LSH	single-wide	lsh, lshadd
LARGEST_SINGLE_WIDE_MUL	single-wide	mul, muladd, muladd2, mulsub, mulsub2
LARGEST_SINGLE_WIDE_DIV	single-wide	divn, divw, divnrem, divwrem
LARGEST_SINGLE_WIDE_ALL	double-wide	the minimum size of the four macros above

We need a mechanism to build and split double-wide types. The lower part of the split is always an unsigned type.

```
S split_upper( DS value );
U split_lower( DS value );
DS wide_build( S upper, U lower );

U split_upper( DU value );
U split_lower( DU value );
DU wide_build( U upper, U lower );
```

The arithmetic functions with an double-wide result are as follows. This category seems less important than the next category.

```
DS wide_lsh( S multiplicand, int count );
DS wide_add( S augend, S addend );
DS wide_sub( S minuend, S subtrahend );
DS wide_mul( S multiplicand, S multiplier );
DS wide_add2( S augend, S addend1, S addend2 );
DS wide_sub2( S minuend, S subtrahend1, S subtrahend2 );
DS wide_lshadd( S multiplicand, int count, S addend );
DS wide_lshsub( S multiplicand, int count, S subtrahend );
DS wide_muladd( S multiplicand, S multiplier, S addend );
DS wide_mulsub( S multiplicand, S multiplier, S subtrahend );
DS wide_muladd2( S multiplicand, S multiplier, S addend1, S addend2 );
DS wide_mulsub2( S multiplicand, S multiplier, S subtrahend1, S subtrahend2 );
S wide_divn( DS dividend, S divisor );
DS wide_divw( DS dividend, S divisor );
S wide_divnrem( S* remainder, DS dividend, S divisor );
DS wide_divnrem( S* remainder, DS dividend, S divisor );

DU wide_lsh( U multiplicand, int count );
DU wide_add( U augend, U addend );
DU wide_sub( U minuend, U subtrahend );
DU wide_mul( U multiplicand, U multiplier );
DU wide_add2( U augend, U addend1, U addend2 );
DU wide_sub2( U minuend, U subtrahend1, U subtrahend2 );
DU wide_lshadd( U multiplicand, int count, U addend );
DU wide_lshsub( U multiplicand, int count, U subtrahend );
DU wide_muladd( U multiplicand, U multiplier, U addend );
DU wide_mulsub( U multiplicand, U multiplier, U subtrahend );
DU wide_muladd2( U multiplicand, U multiplier, U addend1, U addend2 );
DU wide_mulsub2( U multiplicand, U multiplier, U subtrahend1, U subtrahend2 );
U wide_divn( DU dividend, U divisor );
DU wide_divw( DU dividend, U divisor );
U wide_divnrem( U* remainder, DU dividend, U divisor );
DU wide_divnrem( U* remainder, DU dividend, U divisor );
```

The arithmetic functions with a split result are as follows. The lower part of the result is always an unsigned type. The lower part is returned through a pointer while the upper part is returned as the function result. The intent is that in loops, the lower part is written once to memory while the upper part is carried between iterations in a local variable.

```
S split_lsh( U* product, S multiplicand, int count );
S split_add( U* summand, S augend, S addend );
S split_sub( U* difference, S minuend, S subtrahend );
S split_mul( U* product, S multiplicand, S multiplier );
S split_add2( U* summand, S value1, S addend1, S addend2 );
S split_sub2( U* difference, S minuend, S subtrahend1, S subtrahend2 );
S split_lshadd( U* product, S multiplicand, int count, S addend );
S split_lshsub( U* product, S multiplicand, int count, S subtrahend );
S split_muladd( U* product, S multiplicand, S addend1, S addend );
S split_mulsub( U* product, S multiplicand, S subtrahend1, S subtrahend2 );
S split_muladd2( U* product, S multiplicand, S multiplier, S addend1, S addend2 );
S split_mulsub2( U* product, S multiplicand, S multiplier, S subtrahend1, S subtrahend2 );
```

```
S split_divn( S dividend_upper, U dividend_lower, S divisor );
DS split_divw( S dividend_upper, U dividend_lower, S divisor );
S split_divnrem( S* remainder, S dividend_upper, U dividend_lower, S divisor );
DS split_divwrem( S* remainder, S dividend_upper, U dividend_lower, S divisor );

U split_lsh( U* product, U multiplicand, int count );
U split_add( U* summand, U value1, U addend );
U split_sub( U* difference, U minuend, U subtrahend );
U split_mul( U* product, U multiplicand, U multiplier );
U split_add2( U* summand, U value1, U addend1, U addend2 );
U split_sub2( U* difference, U minuend, U subtrahend1, U subtrahend2 );
U split_lshadd( U* product, U multiplicand, int count, U addend );
U split_lshsub( U* product, U multiplicand, int count, U subtrahend );
U split_muladd( U* product, U multiplicand, U multiplier, U addend );
U split_mulsub( U* product, U multiplicand, U multiplier, U subtrahend );
U split_muladd2( U* product, U multiplicand, U multiplier, U addend1, U addend2 );
U split_mulsub2( U* product, U multiplicand, U multiplier, U subtrahend1, U subtrahend2 );
U split_divn( U dividend_upper, U dividend_lower, U divisor );
DU split_divw( U dividend_upper, U dividend_lower, U divisor );
U split_divnrem( U* remainder, U dividend_upper, U dividend_lower, U divisor );
DU split_divwrem( U* remainder, U dividend_upper, U dividend_lower, U divisor );
```

Open Issues

There is more than one possible definition of the sign of the remainder. Can we get by with the one defined above?

Do we want to support all built-in types with these operations, or only the `LARGEST...ALL` plus each of the other `LARGEST` for each operation?